



MASTER BUILD PROMPT — Theme-Driven E-Commerce Platform + God-Mode Admin Panel (Supabase)

0. ROLE, MISSION, AND GROUND RULES

0.1 Your role

You are acting simultaneously as **Lead Software Engineer, Software Architect, DevOps Engineer, QA Engineer, Security Engineer, and Code Reviewer**. You are not a code generator. You are the engineer who will own and maintain this product for the next five years.

You are building a **production-grade, theme-driven e-commerce platform** with a **separate, hardened admin panel**, backed by **Supabase**. The storefront's visual design must match the theme files I provide, pixel-for-pixel. All content inside that design must be database-driven and editable from the admin panel.

0.2 Configuration policy — DO NOT ASK ME FOR SETTINGS

Do not ask me to choose currencies, countries, payment providers, couriers, tax rules, languages, prefixes, or any other configuration value. Every one of those is a runtime setting that lives in the database and is edited in the admin panel. Ship them all, preloaded, enabled where safe, and switchable in the UI. Asking me for a config value that belongs in the admin panel is a design failure.

The only two things I will tell you, and you may ask once if I forget:

1. Project / brand name (used only as the seeded default site title — admin can change it)
2. Primary domain (used only as the seeded default canonical host — admin can change it)

Everything else must be seeded with a working default and made editable in the admin panel:

- **Currencies: seed the complete ISO 4217 list — every currency in the world** (code, name, symbol, decimal places, plural forms), with correct 0-decimal handling (JPY, KRW, VND, CLP, ISK...) and 3-decimal handling (BHD, KWD, OMR, TND, JOD). Admin picks the base

currency and toggles which ones are shown on the storefront. Default: base = USD, all currencies present in the table, a sensible starter set active — **all of it changeable in the panel, no code edit, no redeploy.**

- **Countries:** seed all ISO 3166-1 countries with dial codes, subdivisions/states (ISO 3166-2), and postal-code formats. Admin toggles which ones are shippable.
- **Languages/locales:** seed the locale list; English active by default; admin adds locales and edits every string in the panel (RTL supported).
- **Payment providers:** all adapters shipped in code (Stripe, PayPal, COD, bank transfer, generic hosted-redirect). All disabled until the admin enters credentials and flips the switch.
- **Couriers:** all adapters shipped (Shippo, FedEx, DHL Express, manual). All disabled until credentials are entered.
- **Tax:** per-country/region tax-rate table, empty by default, inclusive/exclusive switch, fully admin-managed.
- **Order prefix:** defaults to `ORD -`, editable in the panel.
- **Timezone, date/number formats, units (metric/imperial), weight units:** seeded defaults, admin-editable.
- **Hosting target:** BOTH. See §20 — the build must run unmodified on Vercel **and** on cPanel/shared Node hosting.

Rule of thumb you must apply everywhere: if a human might ever want to change it, it is a database-backed setting with an admin UI — not a constant, not an env var, not a code edit.

0.3 Attachments I am giving you

1. **Theme files** — HTML/CSS/JS, Figma export, template ZIP, or screenshots. **Read every file before writing a single line of code.** Inventory every page, section, component, breakpoint, font, color token, spacing rule, animation, and icon set.
2. **Invoice template** — the exact invoice design to reproduce for email, on-screen, and PDF.
3. **Admin panel UI reference** — the look and layout of the dashboard.

0.4 Non-negotiable ground rules

- **Never guess.** If a theme file, API contract, or requirement is ambiguous, investigate first. If still ambiguous, list the assumption explicitly in your plan and continue with the safest option.
- **Never hallucinate APIs.** For every third-party integration (Stripe, PayPal, Shippo, FedEx, DHL Express, Supabase), consult the **current official documentation**, pin the exact API

version in code and in `/docs/integrations/<provider>.md`, and record the doc URL and date you read it. If you cannot verify an endpoint or field name, isolate it behind an adapter interface, mark it `// UNVERIFIED:` and flag it in your final report. Do not invent field names.

- **No mock implementations, no TODOs, no stubs, no "left as an exercise".** Every feature listed in this prompt must be fully implemented, wired, and reachable through the UI.
- **No secrets in the repo.** No hardcoded credentials, no keys in client bundles, no keys in git history.
- **Type safety is mandatory.** TypeScript `strict: true`. No `any` without a written justification comment. No `@ts-ignore` without justification. No suppressed lint warnings without justification.
- **Never break existing behaviour.** Additive changes only unless I explicitly ask for removal.
- **Every function:** one responsibility, readable, testable, meaningfully named. No giant components. No duplicated logic. Business logic lives outside UI components.
- **Every error is handled.** No swallowed exceptions. Log with context, fail safely, show a human-readable message, never leak internals to the client.
- **Every important screen handles four states:** loading, empty, error, success.

0.5 Process you must follow (do not skip steps)

For the overall build and for every subsequent change:

1. **Understand** — product purpose, theme structure, data needs, integration constraints.
2. **Analyze** — root cause / affected modules / dependencies / edge cases / security risks / performance risks / scalability.
3. **Plan** — files to create, modify, delete; migrations; risks; test plan.
4. **Implement** — production quality, no shortcuts.
5. **Verify** — run it, test it, prove it.
6. **Refactor** — clean up nearby code where it materially improves maintainability.
7. **Document** — README, `/docs`, inline comments where non-obvious.

0.6 Reporting format for every response you give me

```
## Understanding
## Analysis
## Plan
## Changes (files added / modified / deleted)
```

Risks

Verification (what you ran and what passed)

If you find a better architecture than what I asked for, **explain it before you build it**, then proceed with your recommendation unless I object.

1. TECH STACK (fixed — do not substitute without telling me why)

Layer	Choice	Why
Framework	Next.js (App Router, latest stable), React, TypeScript strict	SSR/ISR/streaming, edge caching, route handlers, server actions
Styling	Tailwind CSS + CSS variables for theme tokens	Theme tokens become DB-driven design tokens
UI primitives (admin)	shadcn/ui + Radix + lucide-react	Accessible, unstyled-first, matches admin reference easily
Backend / DB	Supabase (Postgres + Auth + Storage + Realtime + Edge Functions)	Single source of truth, RLS-based security
Data access	Supabase JS v2 , typed via generated DB types	End-to-end types
Server-side privileged work	Supabase service-role key, server-only (route handlers / server actions / Edge Functions)	Never exposed to browser
Validation	Zod on every boundary (form, API, webhook, env)	No untrusted input
Forms	react-hook-form + Zod resolver	Fast, accessible, typed
Tables (admin)	TanStack Table (server-side pagination/sort/filter)	Scales to 100k+ rows
Images	Custom image API (see §5.4) — Next/Image + Supabase Storage + CDN transform	Speed requirement
PDF (invoices)	React-PDF or Puppeteer/Playwright HTML → PDF in an Edge/Node function	Pixel-match my invoice template
Email	Nodemailer over admin-configured SMTP (§14)	I control the SMTP creds from the panel
Phone validation	libphonenumber-js	Per-country validity
Money	Integer minor units (cents) + <code>decimal.js</code> for math. Never float.	No rounding bugs
Rate limiting	Postgres-backed sliding window + Upstash Redis if available	Brute-force defence

Layer	Choice	Why
Testing	Vitest (unit), Playwright (E2E), pgTAP or SQL tests for RLS	Prove correctness
CI	GitHub Actions: typecheck → lint → unit → build → E2E → migration check	No red main branch
Hosting	Dual target, mandatory: Vercel AND cPanel/shared Node hosting (<code>output: 'standalone'</code> , Node 20+, Passenger). Supabase managed Postgres in both cases. See §20.	I must be able to move hosts without a rewrite
Observability	Sentry (client+server), structured JSON logs, Supabase logs	Debuggable in prod

Monorepo layout

```

/apps
  /web          → public storefront (theme-faithful)
  /admin        → admin panel (separate route group + separate deploy
target)
/packages
  /db           → SQL migrations, generated types, seed, RLS policies
  /core         → domain logic: pricing, cart, tax, shipping, orders,
invoices
  /integrations → payments/*, couriers/*, email/* (adapter pattern)
  /ui           → shared components, design tokens
  /config       → env schema (Zod), constants, feature flags
  /docs         → architecture, runbooks, integration notes, ADRs

```

Admin URL: `/admin` on the main domain by default. Admin and staff both go to `https://yourdomain.com/admin` and sign in with their email + password. This is a hard requirement (it is also what makes single-app cPanel hosting possible).

- The admin is a separate Next.js route group (`app/(admin)/admin/...`) with its own middleware, its own layout, its own cookie name and path scope (`Path=/admin`), and its own strict CSP.
- **Zero admin code in the storefront bundle.** Admin routes are fully code-split; no admin component, query, or type is reachable from a storefront chunk. Verify this with a bundle analyzer and assert it in CI.
- Optional, config-driven: an `ADMIN_HOSTNAME` setting that additionally serves the panel on `admin.yourdomain.com` (cookie scoped to that host). Off by default. `/admin` must always work regardless.
- The admin path can be renamed in settings (e.g. `/manage`) for obscurity, with `/admin` then 404ing. Obscurity is a bonus, never a security control.

2. PRODUCT SCOPE IN ONE PARAGRAPH

A globally fast, fully responsive e-commerce storefront whose entire visual design comes from my theme files, and whose entire content, catalog, pricing, currencies, shipping, payments, branding, SEO, and email configuration are controlled from a single admin panel. Guests can check out without registering; ordering auto-provisions a verified account keyed to a unique email + phone. Customers and admin both receive a beautiful responsive invoice email with a downloadable PDF. Paid orders are automatically pushed to the correct courier (Shippo / FedEx / DHL Express) based on destination country and admin toggles, with a pickup request from the store address. Staff get a restricted view of the same panel — store operations only, zero access to developer/credential settings. Admin changes appear on the live site worldwide **immediately**.

3. THEME FIDELITY RULES (read the theme first)

1. **Inventory pass.** Before coding, produce `/docs/theme-inventory.md` listing: every page, every reusable section, every component, every breakpoint, every font family/weight, the full color palette, spacing scale, radius scale, shadow scale, animation/transition specs, icon set, and third-party libs the theme depends on.
2. **Token extraction.** Convert the theme's palette/typography/spacing into CSS custom properties, stored in a `design_tokens` table and injected server-side as a `<style>` block on first paint (no flash of unstyled content). Admin can edit tokens; changes go live instantly.
3. **Pixel fidelity.** Layout, spacing, and typography must match the theme. Do not "improve" the design. Do not swap fonts. Do not restyle.
4. **Content is data, design is code.** Every string, image, product name, price, badge, banner, menu item, and footer link in the theme becomes a DB-backed, admin-editable value. **No hardcoded copy anywhere.**
5. **Section-based page builder.** Each theme section becomes a registered block type (`hero` , `product_grid` , `banner` , `rich_text` , `testimonials` , `faq` , `newsletter` , ...) with a typed schema. Admin composes pages by adding/reordering/toggling blocks. Renderer maps `block.type` → React component. Unknown block types render nothing and log a warning — never crash.
6. **Responsive at every breakpoint** the theme defines, plus a mobile-first sanity pass at 320px, 375px, 414px, 768px, 1024px, 1280px, 1440px, 1920px.
7. **Accessibility:** semantic HTML, keyboard navigable, visible focus rings, alt text (admin-editable per image), ARIA where needed, target WCAG 2.1 AA. Do not sacrifice theme appearance — fix contrast only where it fails AA, and tell me what you changed.

8. **Visual QA is mandatory.** Render every page in a headless browser, screenshot at **every width listed in rule 12**, compare against the theme reference, and iterate until it matches. Ship the screenshots in `/docs/visual-qa/`, organised per page per breakpoint, and add a Playwright viewport suite plus a visual-regression baseline so a future change cannot silently break a breakpoint.
9. **The nothing-is-hardcoded guarantee.** Apply this test to the theme: *if I can see it in a browser, I must be able to change it in the admin panel*. That includes, and is not limited to — every banner, hero, slider slide, and promo strip; the announcement/marquee bar; header links and mega-menu; every button label; every heading and paragraph of static copy; badges ("Sale", "New", "Free shipping"); trust icons and payment-method logos; newsletter heading, subtext, placeholder, and button; testimonials; FAQ questions and answers; "About us" text; **the entire footer — every column, every link, the newsletter block, social icons and their URLs, contact details, app-store badges, and the copyright / © line including the year and the wording**; cookie-consent text; empty-cart and empty-search messages; 404 and 500 page copy; form labels, placeholders, helper text, and validation messages; loading and success messages; email footer text; `alt` text on every image; page titles and meta descriptions; and the favicon, logo, and every decorative image.
10. **Deliver a content mapping table.** As part of the theme inventory, produce `/docs/content-map.md`: one row per visible string, image, and link found in the theme, giving its location (page → section), what it becomes in the data model (settings namespace, block field, product field, translation key, or nav item), and **the exact admin screen and field where I edit it**. If any row cannot name an admin screen, the build is not finished. Then enforce it: a CI lint rule that fails the build on hardcoded user-facing string literals in components (allowlist only for `aria-label`s and dev-only text), so nobody can quietly reintroduce hardcoded copy later.
11. **How each kind of content is edited** — pick the right home for each, do not dump everything into one giant settings blob:
 - **Banners / sliders / hero / promo strips** → the `banners` table + block builder: image (with separate mobile image), headline, subtext, button label, link target, text position/alignment, overlay opacity, text color, sort order, active toggle, and **schedule (start/end datetime)** so a sale banner appears and disappears on its own. Multiple slides with reorder, per-slide settings, and autoplay/interval controls.
 - **Header, footer, and any menu** → the navigation builder: drag-drop columns and items, nested items, custom labels, internal page/product/category picker or raw URL, open-in-new-tab, optional icon and badge text. The footer builder additionally owns each column's heading, free-text/rich-text blocks, social links, and the copyright line (with a `{{year}}` variable so it never goes stale).

- **Repeating page sections** → block types in the page builder, each with its own typed fields, reorderable, duplicatable, toggleable, and schedulable.
 - **Site-wide chrome and identity** → the `branding` / `seo` / `legal` / `social` settings namespaces (§11).
 - **Interface microcopy** (buttons, validation, empty states, emails) → a translation/string table with a searchable admin editor, so even a single word like "Add to cart" can be reworded per locale without touching code.
12. **Fully responsive on every device — no exceptions.** "Responsive" here does not mean "it doesn't crash on mobile." It means every page, every component, and every admin screen is deliberately designed and verified at every size. Mobile-first CSS: write the small-screen layout first, then layer breakpoints upward.
- **Test at these widths, in both orientations:** 320, 360, 390, 414, 480, 600, 768, 834, 1024, 1180, 1280, 1440, 1920, and 2560 px. Nothing may overflow, clip, overlap, or produce a horizontal scrollbar — **especially at 320 px**, which is the hard floor.
 - **Real device matrix for QA:** iPhone SE, iPhone 15/Pro Max, Pixel 8, Galaxy S24, Galaxy Z Fold (folded *and* unfolded), iPad Mini, iPad Pro portrait + landscape, a 13" laptop, and a 4K desktop.
 - **Touch ergonomics:** every tappable target at least 44×44 px with adequate spacing; no action reachable only by hover (hover states must have tap/focus equivalents); no `:hover` -only dropdowns on touch; swipe gestures for sliders and image galleries; respect iOS/Android safe-area insets (notch, home indicator, punch-hole) via `env(safe-area-inset-*)`.
 - **Layout technique:** fluid grids with CSS Grid/Flexbox, `clamp()` for fluid type and spacing, container queries for components that must adapt to their slot rather than the viewport, `min()` / `max()` instead of fixed pixel widths, and `dvh` / `svh` units so mobile browser chrome doesn't break full-height sections. Never a fixed-width container, never a horizontal-scrolling page.
 - **Media:** every image uses `srcset` + `sizes` served from the image service, with explicit `width` / `height` or `aspect-ratio` to hold space and keep CLS at zero; art-directed mobile crops where the theme's desktop image would lose its subject; videos and embeds wrapped in responsive aspect-ratio containers; background images swapped by breakpoint rather than downscaled.
 - **Storefront mobile patterns:** off-canvas drawer navigation with focus trapping and body-scroll lock; sticky add-to-cart bar on product pages; bottom-sheet filters on category pages; a single-column checkout with the order summary collapsed into an expandable block; a mobile-friendly numeric quantity stepper; the cart as a slide-over.

- **Data-dense UI:** wide tables never just overflow. On small screens they collapse into stacked cards or a labelled key/value list — this applies to the customer's order history and, critically, to the admin panel's order and product tables.
- **The admin panel is responsive too.** A store owner must be able to check orders, update an order status, print/download an invoice, and edit a product from a phone. Sidebar collapses to a drawer, tables become cards, forms go single-column, modals become full-screen sheets, and long forms keep a sticky save bar. Complex bulk-edit screens may be desktop-optimised, but they must still be usable, never broken.
- **Accessibility ties into this:** do not disable pinch-zoom (`user-scalable=no` is banned), stay usable at 200% browser text zoom and with OS-level large text, honour `prefers-reduced-motion` for sliders and animations, honour `prefers-color-scheme` if the theme has a dark variant, and keep full keyboard operability at every breakpoint.
- **Emails and documents count as devices.** Transactional emails use table-based, 600 px-max, mobile-first HTML with inline CSS, tested in Gmail (web/iOS/Android), Apple Mail, and Outlook, with a plain-text alternative and dark-mode-safe colors. The PDF invoice renders correctly at both A4 and US Letter and prints cleanly in black and white.

4. SPEED REQUIREMENTS ("super fast as hell") — measurable targets

These are acceptance criteria, not aspirations.

Metric	Target
LCP (mobile, 4G, p75)	< 1.8s
CLS	< 0.05
INP	< 150ms
TTFB (cached edge hit, any continent)	< 150ms
Storefront JS shipped to browser (initial route)	< 150KB gzipped
Lighthouse mobile (Perf/A11y/Best Practices/SEO)	≥ 95 each
Admin list view, 50k rows, filter+sort	< 400ms server time
Admin edit → visible on live site worldwide	< 3 seconds

How you will achieve it (implement all of these):

- **React Server Components by default.** Client components only where interactivity demands it. No global client-side state library on the storefront unless justified.

- **Static-first rendering with on-demand revalidation.** Every storefront page is statically generated / ISR-cached at the edge with granular cache tags (`product:<id>` , `collection:<slug>` , `settings` , `nav` , `page:<slug>` , `tokens`).
- **Instant global propagation:** on every admin write, a server action calls `revalidateTag()` for the affected tags **and** purges the CDN path. Target: live everywhere in under 3s. Never rely on time-based revalidation alone.
- **Supabase Realtime** for anything that must be live without a page reload: stock counts, cart/checkout availability, admin order board, and a lightweight `settings_version` broadcast so open tabs pick up branding changes without a refresh.
- **Postgres discipline:** indexes on every filter/sort/foreign key column; composite indexes for common admin queries; `pg_trgm` GIN index for product search; materialized views for dashboard aggregates refreshed on write; keyset (cursor) pagination — **never** `OFFSET` **on large tables.**
- **Zero N+1.** Batch with `in()` filters, Postgres views, or RPC functions. Add a dev-mode query counter that fails CI if a page exceeds its query budget.
- **Zero duplicate requests.** Request-level memoization + Next.js `fetch` cache + SWR-style dedupe in the admin.
- **Font strategy:** self-host theme fonts, `font-display: swap` , preload only the fonts used above the fold, subset them.
- **Above-the-fold critical CSS inlined**, rest deferred. No render-blocking third-party scripts; load analytics/chat with `next/script strategy="lazyOnload"` .
- **Route-level code splitting**, dynamic imports for heavy admin widgets (charts, editors, PDF preview).
- **Prefetch** on link hover/viewport for product and collection links.
- **HTTP/2+, Brotli, immutable long-cache headers for hashed assets**, `stale-while-revalidate` for HTML.
- **No layout shift:** every image and embed has explicit width/height or aspect-ratio.
- **Memory-leak audit:** every subscription, interval, and event listener is cleaned up in its unmount/teardown path.
- **Performance budget enforced in CI** — build fails if bundle size or Lighthouse score regresses.

5. ARCHITECTURE

5.1 Layering

```

UI (RSC + client islands)
  ↓
Server Actions / Route Handlers ← Zod validation, authz check, rate limit
  ↓
/packages/core domain services ← pure business logic, unit tested
  ↓
Repositories (Supabase typed client)
  ↓
Postgres + RLS

```

- Domain logic never imports React. UI never writes SQL.
- Integrations are behind **interfaces**: `PaymentProvider`, `CourierProvider`, `EmailProvider`, `StorageProvider`. Adding a provider = adding one adapter file + one settings record. No changes to core.

5.2 Multi-surface auth separation

- **Customers**: Supabase Auth, `role = customer`, cookie scoped to storefront domain.
- **Admin/staff**: Supabase Auth users that also have a row in `admin_users`, cookie named separately and scoped to `Path=/admin` (or the admin hostname if enabled), shorter session TTL, mandatory re-auth for sensitive actions. A user without an active `admin_users` row is rejected at `/admin` even with a valid customer session.
- One Supabase project, two distinct cookies, two middlewares. An admin session must be **useless** on the storefront and a customer session must be **useless** at `/admin`. Prove both with tests.

5.3 Settings architecture (this is what makes everything editable)

Single `settings` table, namespaced key/value with JSONB values and a Zod schema per namespace:

```

branding · seo · theme_tokens · navigation · footer · currency · tax · payments · shipping · couriers ·
smtp · email_templates · sms · orders · security · features · store_profile · legal · social ·
analytics · maintenance

```

- Typed accessor: `getSetting('branding')` returns a fully typed object, cached per-request and edge-cached under tag `settings`.
- Every write bumps `settings_version`, revalidates tags, purges CDN, and broadcasts over Realtime.

- Every write is journaled to `audit_log` with before/after diff and actor.
- **Secrets** (payment keys, courier keys, SMTP password) are stored **encrypted at rest** (pgsodium / Supabase Vault or AES-256-GCM with a key from env), are **never** returned to any client, and are decrypted only inside server-side code. The admin UI shows masked values (`sk_live_****4242`) and a "Replace" action only.

5.4 Dedicated image service (you asked for a separate image API — yes, build it)

- **Storage:** Supabase Storage buckets — `products`, `branding`, `pages`, `invoices` (private), `misc`.
- **Upload API** (`POST /api/images`): server-side only, validates MIME by magic bytes (not extension), enforces max size, strips EXIF, rejects SVG unless sanitized with DOMPurify/ `svgo` allowlist, generates a content-hash filename, and creates responsive derivatives.
- **Derivatives at upload time** via `sharp` : widths `320/480/640/768/1024/1280/1600/1920` , in **AVIF + WebP + JPEG fallback**, plus a 20px blurred **LQIP base64 placeholder** stored on the row.
- **Delivery API** (`GET /api/img/:hash?w=&q=&f=`): signed, cached at the CDN with `immutable, max-age=31536000` , content-negotiated format via `Accept` . Never proxies uncached bytes twice.
- **Row metadata:** `id, bucket, path, hash, width, height, bytes, mime, alt_text, focal_point, lqip, created_by, created_at` . Alt text and focal point are admin-editable.
- **Orphan reaper:** nightly job deletes storage objects with no DB reference (soft-delete window of 30 days first).
- **Frontend:** always `next/image` with correct `sizes` , `priority` only for the hero, `placeholder="blur"` from LQIP.

6. DATABASE SCHEMA (implement all of it, with migrations)

Use SQL migrations in `/packages/db/migrations` , forward-only, each reversible, each idempotent-safe. Generate TS types after every migration. **No destructive migration** without an explicit warning to me and a backup step.

Identity & access

- `admin_users` (id → auth.users, email, name, avatar_image_id, role_id, is_active, must_change_password, last_login_at, password_changed_at, mfa_enabled, created_by)
- `roles` (id, key, name, is_system, description)
- `permissions` (id, key, group, description)

- `role_permissions` (role_id, permission_id)
- `admin_sessions` (id, admin_user_id, ip, user_agent, created_at, last_seen_at, revoked_at)
- `login_attempts` (id, identifier, ip, surface, success, created_at)
- `lockouts` (id, identifier, ip, level, locked_until, reason, created_at)
- `audit_log` (id, actor_type, actor_id, action, entity_type, entity_id, before jsonb, after jsonb, ip, user_agent, created_at)

Customers

- `customers` (id → auth.users nullable, email UNIQUE CI, phone_e164 UNIQUE, phone_country, first_name, last_name, email_verified_at, phone_verified_at, accepts_marketing, default_address_id, created_via ['guest_checkout','signup','admin'], notes, is_blocked, created_at)
- `addresses` (id, customer_id, label, first_name, last_name, company, line1, line2, city, state_region, postal_code, country_code, phone_e164, is_default_shipping, is_default_billing, lat, lng)

Catalog

- `categories` (id, parent_id, name, slug UNIQUE, description, image_id, sort_order, is_active, seo jsonb)
- `brands` (id, name, slug, logo_image_id, is_active)
- `products` (id, title, slug UNIQUE, subtitle, description_rich, short_description, brand_id, status ['draft','active','archived'], is_featured, tags text[], attributes jsonb, seo jsonb, published_at, sort_order, avg_rating, review_count, created_by, updated_by, deleted_at)
- `product_categories` (product_id, category_id)
- `product_images` (id, product_id, image_id, position, is_primary, alt_override)
- `product_options` (id, product_id, name, position) · `product_option_values` (id, option_id, value, position, swatch_hex, swatch_image_id)
- `product_variants` (id, product_id, sku UNIQUE, barcode, option_value_ids uuid[], price_minor, compare_at_price_minor, cost_minor, weight_grams, length_mm, width_mm, height_mm, is_active, position, image_id)
- `inventory` (variant_id PK, quantity, reserved, low_stock_threshold, allow_backorder, track_inventory, warehouse_location)
- `inventory_ledger` (id, variant_id, delta, reason, order_id, actor_id, note, created_at) — every stock change is auditable
- `product_reviews` (id, product_id, customer_id, rating, title, body, status ['pending','approved','rejected'], created_at)

Pricing, promos, currency

- `currencies` (code PK, symbol, name, decimals, is_active, is_base, rounding_rule)
- `exchange_rates` (id, base_code, quote_code, rate numeric(18,8), source, fetched_at)
- `price_overrides` (id, variant_id, currency_code, price_minor, compare_at_minor) — manual per-currency pricing beats auto-conversion
- `coupons` (id, code UNIQUE CI, type ['percent','fixed','free_shipping','bogo'], value, currency_code, min_subtotal_minor, max_discount_minor, applies_to jsonb {all|products|categories|brands}, excludes jsonb, starts_at, ends_at, usage_limit_total, usage_limit_per_customer, first_order_only, combinable, is_active, created_by)
- `coupon_redemptions` (id, coupon_id, order_id, customer_id, amount_minor, created_at)
- `vouchers` / gift cards (id, code UNIQUE, initial_balance_minor, balance_minor, currency_code, issued_to_email, expires_at, is_active, created_by)
- `voucher_transactions` (id, voucher_id, order_id, delta_minor, created_at)
- `offers` (id, name, type ['flash_sale','bundle','tiered','bxgy','free_gift'], config jsonb, priority, starts_at, ends_at, is_active, banner_block_id)
- `tax_rates` (id, country_code, region, rate_bps, is_inclusive, applies_to)

Cart & orders

- `carts` (id, customer_id nullable, session_token, currency_code, coupon_id, voucher_id, shipping_address jsonb, billing_address jsonb, selected_shipping_zone_id, selected_rate jsonb, expires_at, converted_order_id)
- `cart_items` (id, cart_id, variant_id, quantity, unit_price_minor_snapshot, added_at)
- `orders` (id, order_number UNIQUE, customer_id, email, phone_e164, status, payment_status, fulfillment_status, currency_code, presentment_currency, fx_rate_used, subtotal_minor, discount_minor, shipping_minor, tax_minor, total_minor, base_currency_total_minor, coupon_id, voucher_id, shipping_address jsonb, billing_address jsonb, shipping_zone_id, shipping_method_label, courier_provider, customer_note, admin_note, placed_at, paid_at, cancelled_at, ip, user_agent, risk_score)
- `order_items` (id, order_id, variant_id, product_title_snapshot, variant_title_snapshot, sku_snapshot, image_id_snapshot, unit_price_minor, quantity, discount_minor, tax_minor, total_minor) — **snapshots are mandatory**; renaming a product must never mutate a historical order or invoice
- `order_status_history` (id, order_id, from_status, to_status, actor_type, actor_id, note, notify_customer, created_at)
- `payments` (id, order_id, provider, provider_payment_id, method ['card','paypal','wallet','cod','bank_transfer'], amount_minor, currency_code, status,

raw_response jsonb, idempotency_key UNIQUE, created_at)

- **refunds** (id, payment_id, order_id, amount_minor, reason, provider_refund_id, status, actor_id, created_at)
- **shipments** (id, order_id, courier_provider, service_level, tracking_number, tracking_url, label_url, label_image_id, pickup_id, pickup_status, cost_minor, weight_grams, dimensions jsonb, status, shipped_at, delivered_at, raw_request jsonb, raw_response jsonb)
- **shipment_tracking_events** (id, shipment_id, status, description, location, occurred_at, raw jsonb)
- **invoices** (id, order_id, invoice_number UNIQUE, type ['invoice','credit_note'], pdf_path, issued_at, totals jsonb, snapshot jsonb)

Shipping config

- **shipping_zones** (id, name, country_codes text[], match_type ['country','region','postal_prefix','city','custom_area'], match_values text[], priority, is_active)
- **shipping_rates** (id, zone_id, name, calc ['flat','weight_tiered','price_tiered','free_over','carrier_live'], config jsonb, currency_code, min_days, max_days, is_active, sort_order)
- **courier_accounts** (id, provider ['shippo','fedex','dhl_express'], label, credentials_encrypted, mode ['test','live'], is_enabled, priority, capabilities jsonb, last_verified_at)
- **courier_country_routing** (id, country_code, courier_account_id, service_level, is_enabled, priority, auto_create_shipment, auto_request_pickup)

Content & platform

- **pages** (id, slug UNIQUE, title, status, seo jsonb, published_at)
- **page_blocks** (id, page_id, type, position, data jsonb, is_visible, visible_from, visible_until)
- **navigation_menus** (id, key, name) · **navigation_items** (id, menu_id, parent_id, label, url, page_id, product_id, category_id, position, opens_new_tab, icon, badge_text)
- **banners** (id, placement, image_id, mobile_image_id, headline, subtext, cta_label, cta_url, starts_at, ends_at, is_active, position)
- **settings** (namespace, key, value jsonb, is_secret, updated_by, updated_at)
PK(namespace,key)
- **email_templates** (id, key, subject, html_body, text_body, is_active, updated_by)
- **email_log** (id, to_email, template_key, subject, provider_message_id, status, error, order_id, created_at)
- **webhook_events** (id, provider, event_type, external_id UNIQUE, payload jsonb, signature_valid, processed_at, attempts, last_error) — **idempotency table for every inbound webhook**

- `jobs` (id, type, payload jsonb, run_at, attempts, max_attempts, locked_at, locked_by, status, last_error) — durable outbox/retry queue
- `redirects` (id, from_path UNIQUE, to_path, status_code, is_active)
- `search_synonyms`, `newsletter_subscribers`, `contact_messages`, `stock_notifications`

Rules for the schema

- Every table: `id uuid default gen_random_uuid()`, `created_at timestampz default now()`, `updated_at timestampz` maintained by trigger.
- Soft delete (`deleted_at`) for products, categories, customers, pages. Hard delete only for junk (draft carts, expired sessions).
- Foreign keys with deliberate `ON DELETE` behaviour (`RESTRICT` for anything an order references — **never risk data loss**).
- `CHECK` constraints for enums (or Postgres enum types), quantities ≥ 0 , amounts ≥ 0 .
- All multi-step writes (order creation, stock decrement, coupon redemption, payment capture) run inside a **single transaction or a Postgres function** with `SELECT ... FOR UPDATE` on inventory rows. Overselling must be structurally impossible.

7. SECURITY (the admin panel must be "secure as hell")

7.1 Row Level Security

- **RLS enabled on every single table. No exceptions.** Default deny.
- Public/anon: read-only access to `products/variants/categories/pages/page_blocks/banners/settings(non-secret)` where `status='active'` and published.
- Customers: can read/write only rows where `customer_id = auth.uid()`. Cannot read other customers, other orders, or any admin table.
- Admin/staff: access granted through a `has_permission(permission_key)` SQL helper that reads the caller's role permissions. Policies reference the helper, not hardcoded roles.
- Secrets: `settings` rows with `is_secret = true` are readable by **no client role at all** — service role only.
- **Write RLS tests.** A test suite must prove: anon cannot read `admin_users`, anon cannot read `orders`, customer A cannot read customer B's order, staff cannot read payment credentials. CI fails if any of these pass.

7.2 Admin authentication hardening

- Email + password via Supabase Auth at `/admin` (single login form for admins and staff alike — the role decides what they see after login), **plus mandatory TOTP 2FA** for any role with developer permissions, and an admin-controlled "require 2FA for staff" switch. Include recovery codes.
- Passwords: min 12 chars, zxcvbn strength check, breach check against a k-anonymity HIBP range lookup, no reuse of last 5.
- **Progressive brute-force lockout** (exactly what you asked for) keyed on `(identifier, ip)` and separately on `ip` alone:

Consecutive failures	Lock duration
1–4	none (but each failure adds a small artificial delay)
5	15 minutes
6	30 minutes
7	1 hour
8	4 hours
9	12 hours
10+	24 hours + email alert to super admin + require manual unlock or email-link unlock

- Counters decay after 24h of no failures. Lockout enforced **server-side in the database**, not in the client. Return identical generic errors and near-identical response times for "wrong password" vs "unknown user" (no user enumeration). Never reveal remaining attempts precisely — show "Too many attempts. Try again in X minutes."
- Add an invisible **CAPTCHA (Turnstile/hCaptcha)** after 3 failures.
- Global IP-level rate limits on `/admin/login`, password reset, and OTP endpoints.
- **Email + password change from inside the panel** (required): changing either requires current password, then step-up 2FA, then a confirmation link sent to the **old** address for email changes and a verification link to the new one. All active sessions except the current one are revoked on password change. Both events are audit-logged and email-notified.
- **Session security**: `HttpOnly + Secure + SameSite=Lax` cookies, admin cookie scoped to `Path=/admin` (or the admin host), absolute session lifetime 12h (configurable), idle timeout 30m (configurable), session list in the UI with "revoke" and "revoke all others", session invalidation on role change.
- **Step-up re-authentication** required for: viewing/replacing payment or courier credentials, changing SMTP, creating/deleting admin users, changing roles, issuing refunds, bulk

delete, exporting customer data.

7.3 Application security checklist (implement every item)

- **Input:** Zod validate every request body, query param, and webhook payload. Reject unknown keys. Coerce nothing implicitly.
- **Output:** escape everything. Rich text from admin is sanitized server-side with an allowlist sanitizer before storage **and** before render.
- **SQL injection:** parameterized queries / Supabase client only. Never string-concatenate SQL. Any raw SQL lives in reviewed migrations or RPCs.
- **XSS:** strict CSP with nonces (`default-src 'self'` , no `unsafe-inline` for scripts), `dangerouslySetInnerHTML` only on sanitized content with a written justification.
- **CSRF:** double-submit token + Origin/Referer check on every state-changing route; server actions verified.
- **SSRF:** any admin-supplied URL fetch (webhooks, image import, courier endpoints) goes through an allowlist + DNS resolution check blocking private/loopback/link-local ranges and redirects.
- **IDOR:** every read/write authorizes on the resource, never on the ID alone. Use opaque UUIDs, never sequential IDs, in URLs.
- **File upload:** magic-byte MIME check, size cap, extension allowlist, EXIF strip, SVG sanitize, store outside the web root (Supabase Storage), serve via the image API with `Content-Disposition` and `X-Content-Type-Options: nosniff` .
- **Headers:** HSTS (preload), `X-Frame-Options: DENY` on admin, `Referrer-Policy: strict-origin-when-cross-origin` , `Permissions-Policy` minimal, `X-Content-Type-Options: nosniff` , COOP/CORP on admin.
- **CORS:** admin API allows only the admin origin. Storefront API allows only the storefront origin. No wildcards.
- **Webhooks:** verify provider signatures (Stripe `Stripe-Signature` , PayPal transmission verification, courier HMAC) **before** parsing. Store `external_id` uniquely for idempotency. Never trust amounts from the client — always re-read the authoritative amount from the provider API.
- **Rate limits** on: login, register, password reset, OTP, checkout, coupon validation, contact form, search, image upload.
- **Secrets:** env-only, Zod-validated at boot (fail fast), never logged, redaction filter in the logger, `.env` gitignored, rotation runbook documented.

- **PII:** minimal collection, encrypted at rest where sensitive, customer data export + delete endpoints (GDPR), configurable retention for logs.
- **Never store raw card data.** Cards are tokenized by the provider (Stripe Elements / PayPal SDK). The server never sees a PAN. PCI scope stays SAQ-A.
- **Dependency hygiene:** lockfile committed, `npm audit` / Dependabot in CI, no dependency added if the stdlib or an existing dep already solves it.
- **Admin robots:** admin subdomain returns `noindex, nofollow` and is excluded from sitemaps. Optional IP allowlist toggle.

8. CUSTOMER ACCOUNTS, GUEST CHECKOUT & AUTO-PROVISIONING

This is the exact flow you described. Implement it precisely.

1. **Guest can shop and check out with no account.** No forced signup, no interstitial. Cart persists via `session_token` cookie for 30 days and merges into the customer cart on login.
2. At checkout, guest provides **email + phone (with country selector)**. Phone is validated and normalized to **E.164** with `libphonenumber-js`, checked for validity *for the selected country* (not just format), with a live country-flag/dial-code selector defaulting from GeoIP.
3. **Order is placed and paid first.** Never block payment on account creation.
4. **After payment succeeds:**
 - Look up an existing customer by `email` (case-insensitive) **or** `phone_e164`.
 - **If a customer exists** → link the order to that customer and send a "Your order + sign in to track it" email containing a magic link. Do not create a duplicate. Do not silently log the guest in with someone else's account.
 - **If no customer exists** → create a customer row keyed to that unique email + phone, generate a **passwordless email-verification / set-password magic link**, and email it with the order confirmation. The account exists in an `unverified` state; verification activates login.
 - **Conflict case** (email belongs to customer A, phone belongs to customer B) → attach the order to the email match, flag `needs_identity_review` on the order, notify admin, and never merge the two records automatically.
5. **Uniqueness is enforced at the database level** (`UNIQUE` on normalized email and on `phone_e164`), not just in application code.
6. **Login options:** email + password, magic link, or **phone + OTP** (if an SMS provider is configured in settings; otherwise the phone-login button is hidden, not broken).

7. **Customer account area:** orders + statuses + tracking, invoice download, addresses, profile (with verified email/phone change flows), wishlist, saved payment methods (provider tokens only), returns request.
 8. **Emails are enumeration-safe:** "If an account exists, we've sent a link" — always the same response.
-

9. STOREFRONT FEATURES

- Home (fully block-composed), collection/category pages with faceted filtering (price, brand, options, tags, availability, rating) + sorting + keyset pagination, product detail page (gallery with zoom, variant matrix with per-variant price/stock/image, quantity, add-to-cart, wishlist, share, related + recently viewed, reviews, delivery-estimate widget, size guide, stock badge, back-in-stock signup).
 - Instant search with `pg_trgm` typo tolerance, synonyms, debounced suggest dropdown, and a full results page.
 - Cart drawer + cart page: live totals, coupon/voucher entry with clear success/failure reasons, shipping estimator, cross-sell.
 - Checkout: single-page, multi-step accordion. Contact → shipping address (with area/zone selector that instantly reveals the delivery fee) → shipping method (live carrier rates or configured rates) → payment → review. Address autocomplete if a maps key is configured. Full validation, resumable, abandoned-cart capture.
 - Order confirmation page with order number, tracking placeholder, invoice download.
 - **Public order tracking page** `/track?order=<number>&email=<email>` (or tracking number) — no login required, rate limited, shows real courier tracking events on a timeline.
 - CMS pages, blog (optional block), FAQ, contact form, newsletter, legal pages.
 - Currency switcher + country switcher in the header.
 - SEO: per-entity meta from `seo` JSONB, OpenGraph/Twitter cards, JSON-LD (`Product` , `Offer` , `AggregateRating` , `BreadcrumbList` , `Organization` , `WebSite`), canonical URLs, `hreflang` if multi-locale, auto `sitemap.xml` , `robots.txt` , `redirects` table honoured in middleware.
 - Cookie consent banner gating analytics.
 - Maintenance mode toggle (admin bypasses it).
 - Custom 404/500 pages styled to the theme.
-

10. ADMIN PANEL — "GOD MODE", WITH A CLEAN DEVELOPER / STORE SPLIT

10.1 The split you asked for

One panel, two zones, permission-gated. Do **not** build a separate staff website — build one codebase where the sidebar renders only what the signed-in role is permitted to see, and every route re-checks permission server-side. This is less code, fewer bugs, and one place to audit.

Zone	Contains	Who sees it
STORE (day-to-day)	Dashboard, Orders, Shipments, Products, Inventory, Categories/Brands, Customers, Coupons, Vouchers, Offers, Reviews, Content/Pages, Banners, Navigation, Newsletter, Contact messages, Reports	Super Admin, Admin, Manager, Staff (scoped)
DEVELOPER / SYSTEM (locked box)	Site Settings (title, logo, favicon, tokens), SEO defaults, Payment credentials, Shipping & Courier credentials, SMTP & email templates, SMS/OTP provider credentials, Currency & FX, Tax, Order numbering, Security policy, Roles & Permissions, Admin users, Audit log, Webhooks, Jobs queue, Redirects, Feature flags, Import/Export, Backups, Maintenance mode, Integration health	Super Admin + Developer only

The Developer zone is a **visually distinct, collapsed-by-default sidebar group** with a lock icon. Hand a store owner the **Manager** role and they will never see it — the routes 404 for them, the API rejects them, and the nav never renders. Permission checks exist at **three** layers: nav render, server route guard, and RLS policy.

10.2 Roles shipped out of the box (all editable, plus custom roles)

Role	Capability summary
Super Admin	Everything, including roles, admin users, credentials, destructive actions. At least one must always exist (cannot delete the last one, cannot demote yourself into lockout).
Developer	All Developer-zone settings + read on store data. No customer PII export.
Admin / Store Owner	All STORE zone, including refunds, discounts, publishing, customer management. Zero Developer zone.
Manager	STORE zone minus refunds above a configurable threshold, minus customer deletion, minus bulk price edits.
Staff	Exactly what you described: create/edit products, manage inventory, view + confirm + update orders, change order status, create shipments, print/download invoices, create and edit vouchers/coupons, reply to customers. Cannot see revenue reports

Role	Capability summary
	(toggleable), cannot refund, cannot delete anything, cannot touch settings, cannot see payment/courier/SMTP credentials, cannot manage users.
Fulfilment / Warehouse	Orders + shipments + inventory only.
Support	Read orders/customers, add notes, resend emails, request refund (approval required).
Read-only / Auditor	View-only everywhere except secrets.

Permissions are **granular keys** (`product.create`, `product.delete`, `order.refund`, `order.status.update`, `coupon.manage`, `voucher.manage`, `invoice.download`, `settings.payments.read`, `settings.smtp.write`, `customer.export`, `report.revenue.view`, ...) grouped in a checkbox matrix UI. Custom roles are cloneable from any preset. Optional per-user overrides on top of the role.

10.3 Full admin module list (build every one)

1. **Dashboard** — revenue (today/7d/30d/custom, currency-selectable), orders by status, AOV, conversion rate, top products, low stock alerts, failed payments, pending shipments, courier errors, recent activity feed, integration health strip (Stripe  / DHL  / SMTP ). Charts lazy-loaded, aggregates from materialized views.
2. **Orders** — server-side table with saved views and every filter (status, payment status, fulfilment status, date, country, courier, coupon, amount range, search by order number/email/phone/tracking). Bulk actions. Order detail page: items with images, all totals broken out, payment timeline, customer panel, addresses (editable pre-fulfilment), shipping method, courier + tracking, internal notes, full status history, and actions — **change status** (with optional customer notification and note), capture/void, **partial or full refund**, edit line items pre-payment, cancel + auto restock, **download / print / email invoice**, print packing slip, create shipment, buy label, **request pickup**, add tracking manually, resend confirmation, duplicate order, export.
3. **Draft orders** — build an order manually, send a payment link.
4. **Abandoned checkouts** — list + recovery email.
5. **Shipments** — all shipments, tracking status sync, label re-download, pickup status, courier error queue with retry.
6. **Returns / RMA** — request, approve, restock, refund.
7. **Products** — list with inline quick-edit, duplicate, bulk actions (price, status, category, tags). **Product editor with everything on one clean page** (you asked for zero hassle): title, slug (auto with manual override), short + rich description, media (drag-drop multi-upload, reorder, primary, alt text, focal point), pricing (price, compare-at, cost, margin auto-calculated, per-currency overrides), inventory (SKU, barcode, qty, low-stock threshold,

backorder, track toggle), shipping (weight, dimensions, ships-alone flag), **options** → **auto-generated variant matrix** with per-variant price/stock/SKU/image, categories, brand, tags, custom attributes/specs table, related products, SEO panel with live Google preview, status + scheduled publish, and a **live storefront preview**. Autosave drafts. Unsaved-changes guard. CSV/XLSX import with column mapping, validation preview, and dry-run.

8. **Inventory** — stock board, adjustments with reasons, ledger history, low-stock report, incoming stock.
9. **Categories / Brands** — drag-drop tree, images, SEO.
10. **Reviews** — moderate, reply, feature.
11. **Customers** — list, profile with lifetime value + order history + addresses, notes, tags, block/unblock, resend verification, password reset, GDPR export/delete, segments, CSV export.
12. **Coupons** — full builder for every field in `coupons`, live "this is what the customer sees" preview, usage stats, bulk code generation (e.g. 500 unique codes, CSV download), enable/disable, schedule.
13. **Vouchers / Gift cards** — issue, set balance, email to recipient, transaction history, disable, balance lookup.
14. **Offers / Promotions** — flash sales with countdown, BOGO, bundles, tiered discounts, free gift, free-shipping thresholds, priority/stacking rules, auto-generated storefront badges and banners.
15. **Content** — page builder (drag-drop blocks, per-block settings, visibility scheduling, duplicate, preview, publish), navigation builder (mega-menu support), banners/sliders, footer builder, announcement bar, blog.
16. **Media library** — grid, search, folders, alt text, replace, usage list ("used on 3 products, 1 page"), safe delete with usage warning.
17. **Reports** — sales by day/product/category/country/channel, tax report, discount usage, shipping cost vs charged, refunds, inventory valuation, customer cohorts. All exportable to CSV/XLSX.
18. **Notifications** — in-app bell + optional email/Slack/webhook alerts for new order, payment failed, low stock, courier failure, refund, new review.
19. **DEVELOPER ZONE** — everything in §10.1, detailed in §11–§16.

10.4 Multi-admin & staff accounts — exactly how I want it

This is a hard requirement. Build it precisely.

Where: Admin panel → **Settings** → **Team / Users** (Super Admin and Admin only; staff never see this screen).

Creating a staff member (admin does this in ~15 seconds):

1. Admin clicks **"Add user"** and fills a small form: full name, **email** (this is the login username), **password** (admin types it directly, or clicks **Generate** for a strong random one), **role** (dropdown: Admin / Manager / Staff / Fulfilment / Support / Read-only / any custom role), optional per-user permission overrides, and optional notes.
2. On save, the system creates the Supabase Auth user server-side with that email + password, creates the matching `admin_users` row with the chosen role, marks `email_verified` (admin-created accounts do not need to verify — the admin vouched for them), and sets `must_change_password = true` by default (a switch the admin can turn off).
3. The panel shows the credentials **once** in a copy-to-clipboard box ("Email: ... Password: ... Login at: yourdomain.com/admin") and, if SMTP is configured, optionally emails a welcome message with the login URL. **The password is never stored in plaintext and never shown again** — only reset.
4. The staff member goes to `yourdomain.com/admin`, logs in with that email and password, is forced to set a new password on first login (when `must_change_password` is on), then lands on their restricted dashboard.

What the admin can do to a user afterwards: change role, edit per-user permissions, **reset password** (set a new one directly or send a reset link), force logout / revoke all sessions, require 2FA, temporarily suspend (`is_active = false` → immediate login block and session kill), rename, change email (with verification), delete (soft-delete, preserving audit history and their name on past order actions), and view their full activity: last login, IP, device, and every action they took.

Guardrails (implement all):

- Only roles holding `user.manage` can reach this screen. **Staff can never create, edit, view, or delete users — not through the UI, not through the API, not through the database.**
- Nobody can escalate their own role, and nobody can grant a permission they do not themselves hold (no privilege escalation via role editing).
- The last remaining Super Admin cannot be deleted, suspended, or demoted.
- Admin cannot see or set another user's existing password — only overwrite it.
- Every user-management action is audit-logged with actor, target, before/after role and permissions, IP, and timestamp, and emails a notification to the affected user and to Super Admins.
- Creating a user requires step-up re-authentication.

- Password rules from §7.2 apply to admin-set passwords too (length, strength, breach check).
- Optional per-user IP allowlist and login-hours restriction (off by default).

What a Staff user actually sees after login (nothing else — the sidebar renders only these, and every other route 404s for them):

Staff CAN	Staff CANNOT
Simple dashboard: today's orders, pending shipments, low stock, their own tasks	See revenue, profit, margins, or any financial report (admin-toggable per role)
Orders: view, search, confirm, change status , add notes, edit address pre-shipment, create shipment, buy label, request pickup, add tracking, download/print invoice and packing slip, resend confirmation email	Refund, void, capture, delete orders, or edit prices on a placed order
Products: create, edit, upload images, edit descriptions/specs/SEO, manage variants and stock, duplicate	Delete products, bulk-change prices beyond a configurable limit, or publish/unpublish if the admin restricts it
Inventory: adjust stock with a reason, view ledger, low-stock list	Touch inventory valuation or cost prices (cost fields hidden by permission)
Coupons & vouchers: create, edit, disable, look up balances	Exceed a configurable max discount % / max voucher value without approval
Customers: view, search, add notes, resend verification, reply to messages and reviews	Export customer lists, delete customers, or view full payment details
Their own profile: change their own password, enable 2FA, see their own sessions	Anything in the DEVELOPER zone: site settings, logo/title/favicon, payment credentials, courier credentials, SMTP, SMS/OTP credentials, currency/FX, tax, security policy, roles, users, audit log, webhooks, jobs, backups, feature flags

Every "CANNOT" above must be enforced at **all three layers** — the nav does not render it, the server route/action rejects it with 403 (or 404 to avoid disclosure), and the RLS policy refuses the row. A staff member typing a developer URL directly, or calling the API with their own token, must fail. Write an automated test for every single row of this table.

10.5 Admin UX quality bar

Match my admin UI reference. Plus: global command palette (), global search across orders/products/customers, keyboard shortcuts, saved filter views, sticky bulk-action bar, optimistic updates with rollback on failure, toast + inline field errors, skeleton loaders (never a blank flash), unsaved-changes guard on every form, confirm modals with typed confirmation for destructive actions, breadcrumbs, dark mode, full mobile responsiveness (admins do fulfil orders from phones), CSV export on every list, pagination that survives refresh via URL state.

11. SITE SETTINGS & BRANDING (Developer zone)

Everything below is editable in the panel and live within 3 seconds, worldwide.

- **Identity:** site title, tagline, logo (light/dark/mobile variants), favicon (auto-generate the full set: 16/32/48 `.ico`, 180 apple-touch, 192/512 PWA, `manifest.json`, `browserconfig.xml`, maskable icon), OG default image, email logo, loading spinner/brand animation.
- **Design tokens:** all theme colors, fonts (upload or Google Fonts picker with auto self-hosting + subsetting), base font size, radius, spacing scale, shadow scale, container width, button styles, dark-mode palette. Live preview before publish. **Reset to theme defaults** button.
- **Custom code:** head/body script injection slots (nonce-aware, CSP-safe), custom CSS with sanitization, `robots.txt` editor, `sitemap` toggles.
- **SEO defaults:** title template, meta description, keywords, canonical host, indexing toggle, Search Console + GA4/GTM/Meta Pixel/TikTok IDs, structured-data toggles.
- **Store profile:** legal name, store display name, support email, support phone, WhatsApp, **store/pickup address (used as courier from)**, business hours, VAT/registration numbers, social links.
- **Localization:** timezone, date/number formats, default locale, enabled locales, RTL support flag.
- **Legal:** terms, privacy, returns, shipping policy pages; cookie-consent config.
- **Feature flags:** wishlist, reviews, blog, guest checkout, COD, phone login, multi-currency, tracking page, back-in-stock, abandoned-cart emails — each a switch.
- **Order settings:** **order-number prefix (admin-chosen)**, starting number, zero-padding, suffix, per-year reset option, separate invoice-number series with its own prefix. Enforce uniqueness with a Postgres sequence + formatter so concurrent orders never collide.
- **Maintenance mode** with custom message and admin bypass.
- **Global chrome & footer (explicitly admin-owned):** announcement/top bar (text, link, colors, dismissible, schedule), header layout options the theme supports, sticky-header toggle, **footer builder** — every column heading and link, rich-text blocks, contact details, social icons + URLs, payment-method badges, app-store badges, newsletter block copy, and **the copyright line** (editable wording with a `{{year}}` token that auto-updates). Nothing in the header or footer is hardcoded.
- **Banners & sliders:** managed in Content → Banners, per placement (home hero, category top, cart, checkout, sidebar, popup), with desktop + mobile images, headline/subtext/CTA, link, ordering, active toggle, and start/end scheduling.

- **Interface strings:** a searchable string/translation editor for every piece of UI microcopy — button labels, empty states, validation messages, cookie notice, 404/500 copy — per locale.

12. PAYMENTS, CURRENCY & GEO PRICING

12.1 Provider abstraction

```
interface PaymentProvider {
  key: string
  createIntent(order, opts): Promise<{ clientSecret?, redirectUrl?, providerRef }>
  capture(providerRef, amountMinor): Promise<PaymentResult>
  refund(providerRef, amountMinor, reason): Promise<RefundResult>
  verifyWebhook(rawBody, headers): Promise<WebhookEvent>
  supportedCurrencies(): string[]
  supportedCountries(): string[]
}
```

Ship adapters for: **Stripe** (Payment Element → cards, Apple Pay, Google Pay, Link, 3DS/SCA), **PayPal** (Orders v2 + smart buttons), **Cash on Delivery**, **Bank/manual transfer with proof upload**, plus a generic **hosted-redirect adapter** so a regional gateway (SSLCommerz, bKash, Razorpay, Paystack, Mollie...) can be added by writing one adapter file. Read each provider's **current** official docs, pin the API version, and document it.

12.2 Admin control

For each provider: enable/disable toggle, **test/live mode switch**, credential fields (masked, encrypted, write-only), allowed countries, allowed currencies, min/max order value, display name + icon + sort order on checkout, surcharge/discount option, **"Test connection" button** that makes a real read-only API call and reports pass/fail with the actual error message, and webhook endpoint URL + secret shown for copy-paste with a live "last event received" indicator.

12.3 Payment integrity (non-negotiable)

- Amounts are **always** computed and re-verified server-side from the DB cart. Never trust a client-supplied total.
- Idempotency keys on every create/capture/refund. Duplicate submissions cannot double-charge.

- Webhook is the **source of truth** for `payment_status`; the client redirect is only a hint. Handle out-of-order and duplicate webhook delivery via `webhook_events.external_id`.
- Reconciliation job: any order stuck in `pending` for >30 min is re-queried against the provider and resolved.
- Stock is **reserved** at payment intent creation (with expiry) and **committed** on payment success, inside a transaction with row locks. Failed/expired payments release the reservation.

12.4 Multi-currency + automatic geo currency swap (exactly as you described)

- **Every ISO 4217 currency is seeded into the `currencies` table.** Admin sets the base currency from a dropdown of all of them, and toggles `is_active` on any subset for the storefront switcher. Nothing about currency support requires a code change. Correct decimal handling per currency (0, 2, or 3 decimals) is driven by the seeded `decimals` column, never hardcoded to 2.
- Any number of presentment currencies can be active simultaneously.
- **FX rates** auto-fetched on a schedule from a configurable provider (ECB / [exchangerate.host](#) / OpenExchangeRates key in settings), cached, with per-currency **markup %**, **rounding rules** (e.g. round up to nearest 0.99), and **manual override**. If the FX fetch fails, keep the last good rate and alert admin — never show a broken or zero price.
- **Automatic detection order:** explicit user choice (cookie) → GeoIP country (from the CDN's geo header, cached at the edge) → billing/shipping country → phone country dial code → default. When the customer picks their country/phone country at checkout, prices switch to that country's currency automatically.
- Manual **per-variant price overrides per currency** always beat auto-conversion.
- The order stores `currency_code`, `presentment_currency`, `fx_rate_used`, and `base_currency_total_minor` so reporting and refunds are exact forever.
- Refunds are issued in the **original charge currency**.
- If the chosen payment provider does not support the presentment currency, fall back gracefully and tell the customer clearly what they will be charged.
- Currency switcher is edge-cache-safe: vary cached HTML by currency/country segment, or render prices in a small client island hydrated from a cached price payload. **Do not break the CDN cache to do this.**

13. SHIPPING: ZONE/AREA FEES + AUTOMATED COURIER DISPATCH

13.1 Zones, areas, and fees (admin-defined)

- Admin creates **shipping zones** by country, and inside a country by region/state, city, **postal-code prefix**, or a named custom area (perfect for "Inside Dhaka / Outside Dhaka" style rules).
- Each zone holds one or more **rates**: flat fee, weight tiers, price tiers, free over threshold, or **live carrier rates**. Each rate has a label, ETA range (min/max days), and currency.
- **Checkout behaviour**: as soon as the customer selects country → region → area (or types a postal code), the delivery fee appears instantly, along with the ETA. No page reload, no guessing.
- Zone matching is deterministic and priority-ordered: most specific match wins (postal prefix > city > region > country > catch-all). Unmatched destinations show a clear "we don't ship here yet" message plus a configurable fallback.
- Extras: per-product shipping overrides, ships-alone items, dimensional weight, free-shipping coupons, handling fee, insurance option, COD availability per zone, cutoff time + processing days feeding the ETA.

13.2 Courier integrations (Shippo, FedEx, DHL Express) — adapter pattern

```
interface CourierProvider {
  key: 'shippo' | 'fedex' | 'dhl_express'
  validateCredentials(): Promise<HealthResult>
  getRates(shipment): Promise<Rate[]>
  createShipment(order, from, to, parcels, service): Promise<{ tracking
Number, trackingUrl, labelUrl, providerRef }>
  requestPickup(shipmentRefs, pickupWindow, address): Promise<{ pickupId, status }>
  cancelPickup(pickupId): Promise<void>
  track(trackingNumber): Promise<TrackingEvent[]>
  cancelShipment(providerRef): Promise<void>
}
```

Rules for building these adapters:

- Read each provider's **current official API documentation** before writing the adapter. Pin the exact API version. Record the doc URL + date read in `/docs/integrations/<provider>.md`. **Do not invent endpoints or field names.** If a field cannot be verified, mark it `// UNVERIFIED:` and list it in your final report so I can confirm it.
- Handle each provider's real-world quirks explicitly: OAuth token caching and refresh (FedEx, DHL), sandbox vs production base URLs and separate credentials, per-account service-level codes, required shipper/recipient field lengths, country-specific customs/commercial-invoice data for international parcels (HS codes, declared value, contents description — add these as product fields), label formats (PDF/ZPL) and paper sizes, and rate-limit/backoff behaviour.
- **All calls go through the retry queue** (`jobs` table) with exponential backoff + jitter, max attempts, dead-letter status, and a full `raw_request` / `raw_response` audit trail. A courier outage must never lose an order or block checkout.

13.3 Store address & the answer to your question, in code

- Admin sets **store/pickup address + store phone + contact name** once in Store Profile (and optionally multiple warehouse origins, one marked default).
- **The system always sends that address as the shipper/ `from` block on every rate quote, label purchase, and pickup request.** Carriers do not pull it from your account registration — so nothing is manual for you, because the platform injects it automatically every time.
- Validate the store address once via the carrier's address-validation endpoint (where available) and show a green/amber badge in the panel so you know labels will not fail later.

13.4 Per-country courier routing + on/off switches (as you asked)

Admin UI: a routing table, one row per country.

Country	Courier	Service	Auto-create shipment	Auto-request pickup	Enabled
UK	Shippo	Cheapest / specified	☒	☒	☒
USA	FedEx	International Priority	☒	☐	☒
DE	DHL Express	Worldwide	☒	☒	☒
BD	Manual / local	—	☐	☐	☒

Plus: global master switch for automation, per-country priority list with automatic **failover to the next courier** if the first errors, "manual only" mode, and a test-mode switch that logs the exact payload without calling the carrier.

13.5 The automation flow you described, end to end

1. Customer places order → payment webhook confirms `payment_status = paid`.
2. A durable job `dispatch_order_to_courier(order_id)` is enqueued (via the outbox pattern, in the same transaction as the payment update — so it can never be lost).
3. The job **re-verifies from the database that the order is genuinely paid** (never trusts the trigger), and that it is not already dispatched (idempotent on `order_id`).
4. It resolves the courier from `courier_country_routing` using the shipping country (+ zone), respecting enable toggles and priority.
5. It builds the shipment: `from` = **store address + store phone from settings**, `to` = the customer's delivery address + phone, `parcels` = computed from variant weight/dimensions with a packing heuristic, plus customs data for international.
6. `createShipment()` → stores `tracking_number`, `tracking_url`, `label_url` on `shipments`; the label PDF is saved to private Storage.
7. If `auto_request_pickup` is on → `requestPickup()` with the store address and the next valid pickup window (respecting business hours, cutoff time, weekends, and holidays from settings). Stores `pickup_id` + status.
8. Order moves to `processing` → `ready_to_ship` → `shipped`. Customer gets a **"Your order has shipped"** email with the tracking number and link. Admin gets a notification.
9. **Tracking sync:** courier webhooks where supported, plus a scheduled poller for the rest. Events land in `shipment_tracking_events` and drive the public tracking page and status auto-advance (`in_transit`, `out_for_delivery`, `delivered`, `exception`).
10. **Failure path:** on error, retry with backoff; after max attempts, mark the order `courier_failed`, surface it in a **Courier Errors queue** in the panel with the real API error message and a one-click **Retry** and **Switch courier** action, and email the admin. The customer never sees an error.
11. Cancel/refund path: cancel the shipment and pickup with the carrier where the API allows it, restock inventory, void the label.

14. EMAIL, SMTP & INVOICES

14.1 SMTP configured in the admin panel (required)

- Developer zone → Email: host, port, secure (TLS/STARTTLS), username, password (encrypted, write-only), from-name, from-email, reply-to, BCC-all address, envelope sender, per-provider presets (Gmail/Workspace, SendGrid, Mailgun, SES, Postmark, Zoho, custom).

- **"Send test email" button** that reports the real SMTP handshake result and error text.
- If SMTP is not configured, email-dependent features degrade gracefully with a loud admin warning banner — they must not crash checkout. Orders still complete; emails queue in `jobs` and flush once SMTP is valid.
- All sends go through the durable queue with retries, and are logged to `email_log` with status and provider message ID. Failed sends are visible and re-sendable from the panel.
- Deliverability guidance surfaced in the UI: SPF/DKIM/DMARC checklist for the from-domain.

14.2 Transactional emails (all editable templates)

Order confirmation (with invoice) · payment received · payment failed · account created / verify email · magic link · password reset · password changed · order status changed · shipped (with tracking) · out for delivery · delivered · cancelled · refunded · abandoned cart · back in stock · review request · **admin copy of every new order (with invoice)** · admin alerts (payment failed, courier failed, low stock) · staff invitation · contact-form receipt · newsletter welcome.

Template editor: subject + body with a variable picker (`{{order.number}}` , `{{customer.first_name}}` , `{{order.items}}` , `{{tracking.url}}` , ...), live preview with sample data, send-test-to-me, per-template enable toggle, and per-locale variants. Templates are **MJML-based or table-based HTML**, inline-CSS compiled, dark-mode aware, and tested to render correctly in Gmail (web/iOS/Android), Outlook (Windows/web), Apple Mail, and Yahoo. Every email has a plain-text alternative.

14.3 Invoices

- Reproduce **my uploaded invoice template exactly** — layout, typography, spacing, logo placement, table structure, totals block, footer.
- One shared invoice renderer produces three outputs from the same source of truth: **HTML email body, on-screen view, PDF download**. They must be visually identical.
- Contents: store logo + legal details + VAT/registration numbers, invoice number (own admin-set prefix + series), order number, issue date, billing + shipping addresses, line items with variant/SKU/image, unit price, quantity, per-line discount and tax, subtotal, discount (with coupon/voucher code shown), shipping (with method + zone), tax breakdown, grand total, amount paid, balance due, payment method + last-4/PayPal ref, currency clearly stated, shipping/tracking info, notes, terms, thank-you message, and a QR code linking to the tracking page.
- **Immutable snapshot**: the invoice stores its own JSONB snapshot of the order at issue time. Later product renames, price changes, or logo changes must **never** alter a historical invoice.

- **Fully responsive** email invoice (single-column below 480px, no horizontal scroll, no clipped tables).
- Generated asynchronously on payment, stored in a **private** Storage bucket, served via short-lived signed URLs.
- **Download button available to customer, admin, and permitted staff.** Batch download (ZIP) and batch print for admins. Credit notes for refunds. Locale + currency formatting respected.

14.4 SMS / MOBILE-NUMBER OTP VERIFICATION (build it provider-agnostic, off by default)

Phone-number verification by one-time code must be a **first-class, fully built feature** — not a stub — but it must ship **disabled**, cost-aware, and switchable in the admin panel, because every SMS costs real money.

14.4.1 Provider abstraction (same adapter pattern as payments and couriers)

```
interface SmsProvider {
  key: string
  validateCredentials(): Promise<HealthResult>
  sendSms(toE164: string, body: string, opts?): Promise<{ providerRef:
string, segments: number, costMinor?: number }>
  // Optional: providers with a managed verify service (Twilio Verify,
Vonage Verify)
  startVerification?(toE164: string, channel: 'sms' | 'whatsapp' | 'voi
ce'): Promise<{ verificationRef: string }>
  checkVerification?(verificationRef: string, code: string): Promise<{
approved: boolean }>
  supportedCountries(): string[]
}
```

Ship adapters for all of these so I can pick whichever is cheapest in my market without a code change: **Twilio** (raw SMS + Verify), **Vonage/Nexmo**, **MessageBird/Bird**, **Plivo**, **Telnyx**, **AWS SNS**, **Infobip**, **WhatsApp Cloud API** (Meta authentication template), a **generic HTTP adapter** (base URL + method + header/body template with `{phone}` and `{code}` placeholders, so any local/regional aggregator works by filling a form — no code), and a **log adapter** for development that prints the OTP to the server log instead of sending it. Also wire **Supabase Auth phone sign-in** as an option, noting that Supabase does not send SMS itself — it relays through one of the above.

14.4.2 Cost reality — put this in the docs and in the UI, plainly

- **There is no genuinely free, unlimited SMS OTP provider.** SMS delivery is metered by carriers, so every provider either charges per message or gives a small one-off trial credit. Any "free forever SMS" claim is a trial, a tiny daily quota, or unreliable grey-route delivery. State this honestly in the setup guide instead of implying otherwise.
- **What is actually free or near-free**, and must therefore be the default path:
 - **Email OTP / magic link over my own SMTP — genuinely free and unlimited.** This is the default verification channel for account creation, and it must be fully sufficient on its own. The store must be 100% functional with SMS disabled forever.
 - **Trial credits** for a real launch test: Twilio, Vonage, MessageBird, Plivo, and Telnyx all give a small free starting balance — enough to test the whole flow, not enough to run a store.
 - **Small standing free tiers** exist on some platforms (for example AWS SNS's monthly allowance in some regions, and Firebase Phone Auth's small daily quota). Verify the **current** limits from official docs at build time, record them in `/docs/integrations/sms.md`, and never hardcode a number that may have changed.
 - **WhatsApp Cloud API** is often the cheapest per verification and includes a monthly free conversation allowance, but authentication template messages are billed per message in most markets. Build it as an alternative channel with automatic **fallback to SMS** if the number has no WhatsApp.
 - **Local/regional aggregators are almost always dramatically cheaper than global providers** for a single-country store (masked/branded SMS from a domestic gateway typically costs a small fraction of a global per-message rate). This is exactly why the **generic HTTP adapter** exists — I must be able to plug in any local gateway from the panel by entering an endpoint, credentials, and a body template.
- **Show real cost in the panel:** a per-provider cost-per-SMS field I can set, a live counter of OTPs sent today/this month, estimated spend, and a hard **monthly spend cap** that disables SMS OTP and falls back to email when hit. I must never be able to receive a surprise bill.

14.4.3 Where OTP can be required (each one an independent toggle)

Trigger point	Default	Why
Guest checkout phone confirmation	Off	Adds friction and cost to every order. Enable only if fraud demands it.
Cash-on-delivery orders only	Recommended first choice	Verifies exactly the orders that carry real fake-order risk, so spend stays tiny
Orders above a configurable value	Off	Risk-based verification

Trigger point	Default	Why
Account registration / signup	Off (email verification is enough)	Email is free
Login by phone + OTP (passwordless)	Off	Convenience feature; hide the button entirely when no provider is configured
Changing a saved phone number	On when SMS is enabled	Prevents account-takeover via number swap
Admin/staff 2FA fallback	Off — TOTP is preferred	SMS 2FA is the weakest 2FA; never make it the only option for a privileged role
Per-country enable list	Configurable	Skip OTP where delivery is unreliable or expensive

When SMS OTP is disabled or a send fails, the flow must **degrade to email OTP automatically** and never block a sale. A payment must never be lost because an SMS did not arrive.

14.4.4 Implementation requirements (security and abuse are the whole point here)

- Numbers normalised to **E.164** and validated per country with `libphonenumber-js` before a single message is sent. Reject obviously non-mobile line types where the provider can tell us.
- Codes are **6 digits, cryptographically random**, stored **hashed** (bcrypt/argon2) in a `phone_verifications` table — never in plaintext, never in a cookie, never returned in an API response, never logged.
- **Expiry 5 minutes**, max **5 wrong attempts** per code, then the code is burned and a new one must be requested. Verification is single-use and bound to the session and the phone number it was issued for.
- **Resend cooldown** of 60 seconds with a visible countdown, and an escalating cooldown on repeat requests.
- **Rate limits, server-side and in the database** (this is an internet-facing endpoint that spends my money, so treat it like the login form): per phone number per hour, per IP per hour, per session, and a global per-day ceiling. Turnstile/hCaptcha challenge after the second request. Progressive lockout reusing the §7.2 ladder.
- **SMS-pumping / toll-fraud defence** (a real and expensive attack): block premium and known-abused number ranges, block disposable-number prefixes, cap sends per country per day, alert on an abnormal spike or an unusual country mix, and auto-disable sending when the spend cap or spike threshold is hit.
- The message body is an **editable template** (`sms_templates`, per locale) with a variable picker, a sender-ID/brand-name field where the country allows it, and a character/segment

counter showing the real cost of the message.

- Every send is written to an `sms_log` table (recipient masked as `+8801*****34`, template, provider, provider ref, segments, cost, status, error) and is visible and re-sendable in the panel, exactly like `email_log`.
 - **"Send test SMS" button** in the panel that performs a real send and reports the provider's actual response and error text.
 - Delivery receipts/webhooks consumed where the provider supports them, so a failed delivery is visible rather than assumed successful.
 - On success, set `customers.phone_verified_at`, show a verified badge in the admin customer profile, and audit-log it.
 - Accessibility and UX: `autocomplete="one-time-code"` on the input, `inputmode="numeric"`, auto-advance and paste-friendly single input, iOS/Android SMS autofill working, a clear "didn't get it? resend / try email instead / change number" path, and full keyboard operability.
 - Provider credentials live **encrypted in the settings table, entered in the panel** — never in env, never in the repo. Test/live mode switch like every other integration.
-

15. ORDER LIFECYCLE, NUMBERING & TRACKING

- **Order numbering:** `{{prefix}}{{padded_sequence}}{{suffix}}` — prefix chosen by admin (seeded default `ORD-`), backed by a Postgres sequence inside the insert transaction so it is unique and gapless-by-design under concurrency. Optional yearly reset (`ORD-2026-000123`). Invoice numbers are a **separate** series. Never expose a guessable ID in a public URL without also requiring the email (tracking page).
- **Statuses** (separate, orthogonal fields — do not cram them into one):
 - `status`: `pending → confirmed → processing → ready_to_ship → shipped → out_for_delivery → delivered → completed`, plus `on_hold`, `cancelled`, `failed`, `returned`.
 - `payment_status`: `unpaid → authorized → paid → partially_refunded → refunded → failed → voided`.
 - `fulfillment_status`: `unfulfilled → partially_fulfilled → fulfilled → returned`.
- **Admin/staff can change status** from the order page or in bulk, with an optional note and a "notify customer" checkbox. Illegal transitions are blocked by a state machine in `/packages/core`, not by UI hope. Every transition is written to `order_status_history` with actor and timestamp.
- **Tracking must actually work:** real tracking number from the carrier, correct carrier-specific tracking URL, a branded public tracking timeline built from `shipment_tracking_events`, webhook + poller sync, multi-parcel support, manual tracking entry for manual couriers, and "exception" handling that alerts admin and shows the customer a helpful message.

Test with each carrier's sandbox tracking numbers and include the results in your verification report.

- **Partial fulfilment:** split shipments per order, each with its own tracking.
 - **Restock on cancel/refund** through `inventory_ledger`, never by a bare `UPDATE`.
-

16. OBSERVABILITY, JOBS & OPS

- **Job queue UI:** pending / running / failed / dead-letter, with payload inspector, retry, and cancel.
- **Webhook inspector:** every inbound event, signature validity, processing result, replay button.
- **Integration health page:** live credential validation for every payment provider, courier, SMTP, and FX source, with last-success timestamps.
- **Audit log viewer:** filter by actor, action, entity, date; before/after JSON diff; export. Immutable (append-only, no UPDATE/DELETE policy).
- **Structured JSON logging** with request IDs correlated across storefront → server action → Supabase → external API. Secrets redacted by the logger.
- **Sentry** for client + server, with release tagging and source maps (not publicly served).
- **Uptime + error alerting** to the admin email/Slack webhook.
- **Backups:** see §16.1 below — this is a first-class feature, not a footnote.
- **Migration safety:** CI checks migrations apply cleanly to a fresh DB *and* to a copy of production schema. No destructive step without an explicit guard.

16.1 BACKUPS, EXPORTS & DATA SAFETY — my data must be impossible to lose

Treat this section as seriously as payments. I will have thousands of products, real customers, and real orders. **Losing them is a catastrophic, unrecoverable failure.**

16.1.1 The architectural guarantee (state this in the docs in plain language)

- **Code and data are completely separate.** The application code lives in the repo and on the host (Vercel/cPanel). All data — products, customers, orders, invoices, settings — lives in Supabase Postgres, and all uploaded files live in Supabase Storage. **Redeploying, replacing, or rewriting the application does not touch the database or storage.** Explain this explicitly in `/docs/SETUP.md` so I never have to worry about it.

- **Migrations are forward-only and additive.** Adding a feature adds tables/columns; it never drops or truncates existing data. Any migration that would drop a column or table, change a type destructively, or delete rows must: (a) be clearly labelled `DESTRUCTIVE` in its filename and header comment, (b) refuse to run unless an explicit `ALLOW_DESTRUCTIVE=1` flag is set, (c) **take an automatic backup first**, and (d) be reported to me in advance with the exact data at risk. There is no "drop and recreate" path in this project, ever.
- **No deploy step may run `db:reset`, `supabase db reset`, `TRUNCATE`, or a seed that overwrites existing rows.** The seed script must be strictly idempotent and insert-if-absent only — running it against a live database with 1,200 products must change nothing. Guard it with a row-count check and a hard refusal in production.
- **Data outlives the host.** Because state lives in Supabase, I can move from cPanel to Vercel (or to a VPS) by pointing the new deployment at the same Supabase project. Document this migration path.

16.1.2 Automatic backups (all three layers, always on)

1. **Point-in-time recovery (PITR)** on the Supabase project — restore to any second within the retention window. Document how to enable it and what plan it needs.
 2. **Nightly full logical dump** — a scheduled job runs `pg_dump` (custom format, compressed), plus a schema-only dump, and uploads them **off-site** to a destination I configure in the panel (S3 / Cloudflare R2 / Backblaze B2 / Google Drive / SFTP). Off-site matters: a backup stored only inside the same Supabase project is not a backup.
 3. **Storage/media backup** — the `products`, `branding`, `pages`, and `invoices` buckets are synced to the same off-site destination with a manifest file, because a database restore without images is a broken store. Incremental after the first full sync.
- **Encryption:** every backup artifact is encrypted at rest (AES-256, key from env, documented rotation). Backup files must never be publicly reachable.
 - **Retention policy, admin-configurable:** 7 daily + 4 weekly + 12 monthly by default, with automatic pruning. Show total size and next-run time in the panel.
 - **Integrity checks:** after each dump, verify the archive is readable (`pg_restore --list`), record checksum, row counts per critical table, and byte size. A dump that shrinks by more than a configurable % versus the previous run raises a loud alert — that is how silent corruption gets caught.
 - **Monitoring:** if a scheduled backup does not complete, email/alert me. A missed backup must be **noisy**, never silent. Show a red banner in the dashboard after 48h without a verified backup.
 - **Works on both hosts:** on Vercel the schedule is a cron route; on cPanel it is one of the supplied cron lines (§20.1). Same code path.

16.1.3 Backups screen in the admin panel (Developer zone)

A real UI, not just scripts:

- **"Back up now"** button — runs a full backup on demand and shows live progress.
- **Backup history table** — timestamp, type (auto/manual/pre-migration), size, contents (DB / media / both), integrity status, destination, and a **download** action for each.
- **"Download everything"** — produces one archive containing the SQL dump, the media files (or a signed manifest for very large stores), and a `settings.json`, so I always hold a complete copy of my own store on my own laptop.
- **Destination configuration** with a **"Test connection"** button that actually writes and deletes a probe file.
- **Restore** — restoring is deliberately guarded: super-admin only, step-up re-auth, typed confirmation, a mandatory pre-restore safety backup taken first, and a clear preview of what will be replaced. Never a one-click button that can nuke production by accident.
- **Per-entity exports** — CSV/XLSX/JSON export for products (with variants, images, inventory), customers, orders (with line items), invoices, coupons, vouchers, reviews, pages, and settings. Available as one-off downloads and as **scheduled exports** emailed or pushed to cloud storage. Matching **importers** exist for products and customers (with column mapping, validation preview, and dry-run), so a full rebuild from exports is possible even without SQL access.

16.1.4 The safe-upgrade playbook (write this as `/docs/runbooks/upgrade.md`)

Because I *will* want to change or replace the code later, deliver an explicit procedure and prove it works:

1. Take a manual backup and confirm it verified.
 2. Restore that backup into a **staging Supabase project** (the guide includes the exact commands).
 3. Deploy the new code against staging; run migrations there first.
 4. Run the automated suite plus a smoke test on staging with real-shaped data.
 5. Only then deploy to production, run migrations, and watch Sentry + the health page.
 6. **Rollback plan:** the previous deployment is kept and re-promotable (Vercel instant rollback / a kept `previous-release` directory on cPanel), with documented steps and the fact that data is untouched by a code rollback.
- Every migration ships with a written **reverse migration** or a documented reason one is impossible.

- The app records its schema version and refuses to boot against an incompatible database with a clear message instead of corrupting data.

16.1.5 Restore drill (this is the part everyone skips — do not skip it)

- `/docs/runbooks/restore.md` with exact, tested commands for: full restore, single-table restore, single-row recovery from PITR, and media-only restore.
- **You must actually perform a full restore drill into a scratch project before calling the project done**, and report: time taken, row counts before/after, and screenshots of the restored store running. Include a scenario test — "1,200 products and 3,000 orders restored intact, images loading, invoices still matching originals."
- Soft delete plus a 30-day trash/undo for products, categories, customers, and pages, so ordinary human mistakes never need a restore at all. Storage objects get the same 30-day grace before the orphan reaper touches them.
- Order, invoice, payment, and audit-log rows are **append-only in practice**: no admin UI path deletes them, only cancel/refund/credit-note flows that preserve history.
- **Seed script** that creates the Super Admin, default roles/permissions, currencies, one shipping zone, sample products, and demo content — so the app is usable within 60 seconds of setup.
- `/docs` : architecture overview, ERD, ADRs, environment setup, deployment guide, integration notes (with doc URLs and API versions), permission matrix, runbooks (courier failure, payment mismatch, restore, key rotation), and a troubleshooting guide.

17. TESTING & DEFINITION OF DONE ("I don't want any bugs")

You may not tell me a feature is complete until all of the following pass and you show me the output.

Automated

- `tsc --noEmit` clean. ESLint clean. Prettier clean.
- **Unit tests** for all domain logic: pricing, discount stacking, coupon eligibility, voucher balance, tax, FX conversion + rounding, shipping-zone matching, weight tiers, order-number formatting, state-machine transitions, phone normalization per country, invoice totals. Money math tested for rounding at boundaries.
- **RLS/authz tests** proving each negative case in §7.1 and the full permission matrix (staff blocked from every developer route and API).

- **Integration tests** against provider sandboxes: Stripe (success, decline, 3DS challenge, refund, duplicate webhook, out-of-order webhook), PayPal (approve, cancel), Shippo/FedEx/DHL (rate, label, pickup, track, error, timeout, retry).
- **E2E (Playwright)** covering the full journeys: guest checkout → payment → order → invoice email → auto account creation → verification → login → order visible; registered checkout; coupon + voucher + free-shipping; currency swap by country; area-based delivery fee; admin login with lockout escalation; admin edits title/logo/favicon → live within 3s; product create with variants → visible on storefront; order status change → customer email; refund → credit note; staff login → developer routes 404. Run on Chromium, Firefox, WebKit, and mobile viewports.
- **Concurrency test:** 50 simultaneous checkouts on 1 unit of stock → exactly 1 succeeds, 0 oversells.
- **Load test:** k6/Artillery on storefront and checkout; report p95 latencies.
- **Lighthouse CI** enforcing \$4 budgets. **axe-core** accessibility scan with zero critical violations.
- **Visual regression** screenshots vs the theme reference at every breakpoint.

Manual verification you must perform and report on

- Every admin form: save, validate, error, permission-denied, and audit-log entry.
- Email rendering across the clients listed in §14.2.
- Invoice PDF vs my template, side by side.
- Offline/slow-network behaviour, expired session behaviour, double-submit behaviour, browser back-button after payment.
- Security spot-checks: try IDOR on an order, try a staff account against a developer API route, try an unsigned webhook, try SQL/XSS payloads in every input, try uploading a PHP file renamed to `.jpg`.

Debugging rule: when something breaks, trace `Problem → Cause → Dependency → Fix → Verification`. Fix the root cause. **Never patch a symptom.** Never silence an error to make a test pass.

18. DELIVERY PLAN (build in this order, ship working software at every step)

Phase	Deliverable	Gate
0	Theme inventory doc, ERD, architecture doc, implementation plan, risk list	I approve before code

Phase	Deliverable	Gate
1	Monorepo, env schema, Supabase project, migrations, RLS + RLS tests, seed, CI pipeline	CI green
2	Design tokens + theme-faithful component library + visual QA	Screenshots match
3	Storefront read paths (home, collection, PDP, pages) with ISR + tag revalidation	Lighthouse ≥95
4	Admin shell, auth, roles/permissions, brute-force lockout, audit log, settings framework	Authz tests pass
5	Products/variants/inventory/media + image API	Create → live in <3s
6	Cart, checkout, phone validation, guest flow, auto account provisioning	E2E green
7	Payments (Stripe, PayPal, COD) + webhooks + idempotency + refunds	Sandbox suite green
8	Currency, FX, geo swap, per-currency overrides, tax	Money tests green
9	Shipping zones/areas/rates + checkout fee reveal	Zone tests green
10	Courier adapters + auto dispatch + pickup + tracking + error queue	Sandbox suite green
11	SMTP, all email templates, invoice HTML/PDF	Client-render matrix
12	Coupons, vouchers, offers, reviews, wishlist, search	E2E green
13	Content/page builder, navigation, banners, SEO, redirects, sitemap	SEO audit clean
14	Reports, dashboards, notifications, jobs/webhook inspectors, health page	Manual pass
15	Hardening pass, load test, a11y pass, docs, runbooks, backup/restore drill, launch checklist	All gates green

After every phase, give me the \$0.6 report plus a short demo script telling me exactly what to click to verify it.

19. ENVIRONMENT VARIABLES (validate with Zod at boot; fail fast if missing)

```
# --- required everywhere ---
NEXT_PUBLIC_SITE_URL           # https://yourdomain.com
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY     # server only, never bundled
DATABASE_URL                   # migrations only
```

```

APP_ENCRYPTION_KEY          # 32-byte key for settings-secret encryption
REVALIDATION_SECRET
CRON_SECRET

# --- deployment target ---
DEPLOY_TARGET=vercel|node    # switches cache handler, cron, geo, image strategy
PORT                         # cPanel/Passenger only
CACHE_DIR=/home/user/app-cache # cPanel only: filesystem ISR cache path

# --- optional ---
ADMIN_HOSTNAME               # optional extra admin host; /admin always works
SENTRY_DSN / SENTRY_AUTH_TOKEN
TURNSTILE_SITE_KEY / TURNSTILE_SECRET_KEY
UPSTASH_REDIS_REST_URL / _TOKEN # optional; Postgres fallback if absent
MAXMIND_LICENSE_KEY          # optional GeoIP fallback when no CDN geo header

```

That is the **entire** env surface, and none of it is business configuration.

Everything else — Stripe, PayPal, Shippo, FedEx, DHL, SMTP, FX provider, maps, analytics IDs — is **admin-configurable in the panel and stored encrypted in the database, not in env**. That is the point: I must be able to change credentials without a redeploy.

20. HOSTING: MUST RUN ON VERCEL AND CPANEL

This is a hard requirement, not a nice-to-have. One codebase, one build, two deploy targets, identical features. Nothing may depend on a Vercel-only primitive without a working Node fallback selected by `DEPLOY_TARGET`.

20.1 Portability rules

- **Build output:** `next.config.js` sets `output: 'standalone'` so the app runs as a plain Node server (`node server.js`) on any host. Vercel ignores it harmlessly.
- **No edge runtime requirements.** Middleware and all route handlers must run correctly in the **Node.js runtime**. Do not use `export const runtime = 'edge'` anywhere that matters.

- **Abstract every platform capability behind an interface with two implementations,** chosen at boot by `DEPLOY_TARGET` :

Capability	On Vercel	On cPanel / Node
ISR / page cache	Native ISR + <code>revalidateTag</code> • CDN purge	Custom <code>cacheHandler</code> writing to <code>CACHE_DIR</code> on disk (or Redis if configured), plus <code>stale-while-revalidate</code> response headers so Cloudflare/LiteSpeed caches for you. <code>revalidateTag</code> must invalidate the disk cache and, if a CDN token is set, call its purge API.
Cron jobs	<code>vercel.json</code> cron → <code>/api/cron/*</code>	cPanel → Cron Jobs calling <code>curl -s -H "x-cron-secret: ..." https://site/api/cron/<job></code> . Ship the exact crontab lines in the setup guide. Also ship a self-scheduling in-process worker fallback for hosts without cron.
Background jobs (emails, courier dispatch, invoices)	Cron-driven queue worker	Same queue, drained by the cron endpoint and/or a long-running Passenger worker. The <code>jobs</code> table is the queue in both cases — never an in-memory queue.
Image optimization	Next/Image optimizer	Derivatives are pre-generated with <code>sharp</code> at upload (\$5.4), so no runtime optimizer is needed. Set <code>images.unoptimized</code> only if <code>sharp</code> is unavailable; the pre-generated AVIF/WebP set still serves.
GeoIP for currency	<code>x-vercel-ip-country</code>	<code>cf-ipcountry</code> (Cloudflare), then <code>CF-IPCity</code> / LiteSpeed headers, then optional MaxMind lookup, then default. One resolver function, four sources, never a crash.
Rate limiting	Upstash Redis or Postgres	Postgres sliding window (already the default). No dependency on any Vercel KV.
Logs	Vercel logs + Sentry	Rotating JSON files under the app dir + Sentry. Log path configurable.
Webhooks (Stripe/PayPal/courier)	Route handler	Identical route handler. Raw-body access must work under Passenger — verify signature verification on both targets.

- **No filesystem writes anywhere except the cache/log dirs.** All user content lives in Supabase Storage, so cPanel disk limits and Vercel's read-only FS both work.
- **Long-running work must be chunked** so it finishes inside typical shared-host and serverless timeouts (bulk import, bulk label buy, CSV export → queue + progress UI, never a 5-minute request).
- **CI must build and smoke-test both targets:** a Vercel build and a `standalone` Node build booted in a container with `DEPLOY_TARGET=node`, running the same E2E suite. A PR that breaks cPanel compatibility fails CI.

- Deliver `Dockerfile` + `docker-compose.yml` as a third, host-agnostic option (VPS), plus `ecosystem.config.js` for PM2.
- Document the honest trade-off in `/docs/hosting.md`: cPanel gives you full control and cheap hosting but a single origin (slower TTFB for distant users unless you front it with Cloudflare — which the setup guide tells me how to do), while Vercel gives global edge automatically. **Feature parity must be 100% either way.**

20.2 cPanel-specific requirements

- Must work with **cPanel** → **Setup Node.js App** (Passenger): application root, application URL, startup file `server.js`, Node 20+, `NODE_ENV=production`.
- Ship a `server.js` entry that boots the standalone Next server on `process.env.PORT`.
- Ship the `.htaccess` needed for static asset caching and HTTPS redirect, and document LiteSpeed/Apache specifics.
- Ship `scripts/deploy-cpanel.sh` that builds locally, produces a deployable bundle (`.next/standalone` + `.next/static` + `public`), and lists the exact upload/restart steps.
- Zero native-module surprises: `sharp` must install on the host's Node ABI, with a documented fallback if it cannot (`--platform` prebuilt binaries, or pre-generate derivatives during the build/upload step).
- Works on a subdirectory install too (`basePath` configurable), though root domain is the default.

21. SETUP GUIDE (deliver this as `/docs/SETUP.md` — write it for a non-expert)

Produce a complete, copy-paste, screenshot-friendly guide that takes someone from zero to a live store. It must be accurate — you must actually run every step yourself and fix the guide where reality differs. Required structure:

1. Prerequisites — Node 20+, pnpm, a Supabase account, a domain, and (for cPanel) a host with the Node.js App feature and Node 20+. Exact version numbers.

2. Get the code running locally

- Clone, `pnpm install`, copy `.env.example` → `.env.local`.
- Create the Supabase project, where to find each key in the dashboard (exact menu path), what each key does, and which one must never leave the server.
- `pnpm db:migrate` → `pnpm db:seed` → `pnpm dev`.

- **Create the first Super Admin:** `pnpm create-admin --email you@example.com` (interactive password prompt). Log in at `http://localhost:3000/admin`.
- What the seed gives you: all currencies, all countries, default roles and permissions, email templates, one shipping zone, demo products, demo pages.

3. First-run wizard in the panel — a guided checklist the app itself shows on first login, with a progress bar, so I am never guessing what is left: store profile & pickup address → branding (title, logo, favicon) → base currency + active currencies → tax → shipping zones & fees → payment provider credentials → SMTP + test email → courier credentials + per-country routing → order prefix → invoice check → security (2FA, logout) → add staff → go live. Each item deep-links to the right settings screen and marks itself done.

4. Deploy to Vercel — import repo, set env vars (exact list, which are public vs secret), configure `vercel.json` crons, add the domain and DNS records, set the payment webhook URLs in Stripe/PayPal dashboards, verify with a test order.

5. Deploy to cPanel — step by step with exact clicks:

- Build locally (`pnpm build:standalone`), what to upload and where.
- cPanel → Setup Node.js App: application root, URL, startup file, Node version, environment variables, then Restart.
- Install dependencies via the cPanel "Run NPM Install" button or terminal.
- The `.htaccess` to add and why.
- **The exact cron lines to paste into cPanel → Cron Jobs**, with schedules, for: job queue drain (every minute), FX rate refresh (hourly), courier tracking poll (every 15 min), abandoned-cart emails (hourly), sitemap regen (daily), orphan image reaper (nightly), backup (nightly), stuck-payment reconciliation (every 10 min).
- SSL via AutoSSL/Let's Encrypt.
- Optional but recommended: putting Cloudflare in front for global speed and how to configure cache rules so admin edits still appear instantly (bypass rules for `/admin` and `/api`, purge-on-write via API token).
- Restart / rollback / log locations / "it shows a 503, what now" troubleshooting.

6. Configure the store (panel walkthrough) — one short section per settings screen, with a screenshot placeholder and "what happens if you get this wrong": branding, currencies & FX, tax, shipping zones and area fees, payment providers (with where to get each key and where to paste the webhook URL), SMTP (with Gmail/SendGrid/SES specifics and the SPF/DKIM/DMARC records to add), couriers (how to get Shippo/FedEx/DHL API credentials, sandbox vs live, per-country routing table), order numbering, invoice check, SEO & analytics.

7. Add your team — the exact click path to create a staff account, what each role can do (paste the §10.4 permission table), how to reset a password, how to suspend someone who leaves,

and how to read the audit log.

8. Day-to-day operations — add a product with variants, run a discount/voucher, process an order end to end (confirm → label → pickup → shipped → delivered), issue a refund, handle a courier failure from the error queue, edit the homepage with the block builder.

9. Maintenance & safety — how backups work and **how to actually restore one** (tested, with the commands), how to rotate keys, how to update the app safely, how to read Sentry, and a one-page "something is broken" triage flowchart.

10. Go-live checklist — printable, covering: test mode off everywhere, real payment test with a real card refunded, SMTP verified from a real inbox, invoice checked on mobile, SSL + HSTS live, `noindex` removed from the storefront and kept on `/admin`, sitemap submitted, 2FA on for every admin, seed/demo data removed, backups confirmed running, and the Super Admin recovery codes stored somewhere safe.

Also deliver a short `README.md` (what this is, how to run it, links into `/docs`) and `/docs/ADMIN-GUIDE.md` written for a non-technical store owner, in plain language, no jargon.

22. FINAL ACCEPTANCE CHECKLIST

Tick every line and paste the ticked list back to me with evidence.

- ☐ Storefront visually matches the theme at every breakpoint (screenshots attached)
- ☐ Zero hardcoded content — every string, image, price, and link is admin-editable
- ☐ Any admin change is visible worldwide in under 3 seconds
- ☐ Lighthouse mobile ≥95 across all four categories on home, collection, PDP
- ☐ Guest checkout works end to end without an account
- ☐ Order auto-creates a verified account keyed to unique email + phone; existing email/phone logs into the existing account instead of duplicating
- ☐ Phone numbers validated per country and stored as E.164
- ☐ Customer **and** admin both receive the order email with the pixel-matched responsive invoice + PDF
- ☐ Invoice downloadable by customer, admin, and permitted staff; historical invoices immutable
- ☐ Currency switcher works; prices auto-swap to the customer's country currency; refunds in original currency
- ☐ Delivery fee appears immediately when an area is selected; zone/area/country rules configurable in the panel

- ☐ Paid orders auto-dispatch to the correct courier per country, with the store address auto-injected as shipper, and pickup auto-requested
- ☐ Courier failures are queued, retried, visible, and retryable — never silent, never blocking
- ☐ Tracking numbers are real and the public tracking timeline updates from carrier events
- ☐ Order status changeable by admin/staff with history + optional customer notification
- ☐ Order-number prefix configurable; numbers unique under concurrency
- ☐ Admin can change their own email and password securely from inside the panel
- ☐ Brute-force lockout escalates 15m → 30m → 1h → 4h → 12h → 24h, enforced server-side
- ☐ 2FA available and enforced for developer-level roles
- ☐ Staff role sees only store operations; every developer route is invisible **and** returns 403/404 for them
- ☐ All credentials (payments, couriers, SMTP) encrypted at rest, masked in UI, never sent to any client
- ☐ RLS enabled on every table with negative tests proving isolation
- ☐ No overselling under 50-way concurrent checkout
- ☐ Webhooks signature-verified and idempotent
- ☐ Zero TypeScript errors, zero lint errors, zero TODOs, zero mocks, zero **any** without justification
- ☐ CI green: typecheck, lint, unit, RLS, integration, E2E, Lighthouse, a11y
- ☐ Docs, ERD, permission matrix, runbooks, and a tested restore drill delivered
- ☐ **All ISO 4217 currencies seeded and selectable in the panel; 0/2/3-decimal currencies all render and calculate correctly**
- ☐ **All ISO 3166 countries + subdivisions + dial codes seeded; shippable countries toggled in the panel**
- ☐ **No business configuration lives in code or env — I can change every setting from the admin panel with no redeploy**
- ☐ **Admin creates a staff account (email + password + role) in the panel; staff logs in at `/admin` and is forced to set a new password**
- ☐ **Staff sees only the store features; every developer route and API is blocked at nav, server, and RLS layers — tested per row of the §10.4 table**
- ☐ **Staff cannot create, edit, or view other users; no privilege escalation is possible; last Super Admin cannot be removed**

- ☐ Admin can reset a staff password, revoke sessions, suspend, and audit every action they took
- ☐ The identical build runs on Vercel AND on cPanel (Node.js App / Passenger) with 100% feature parity — both proven in CI and manually
- ☐ cPanel deployment works with the supplied cron lines; queue, FX, tracking poll, and emails all run
- ☐ `/docs/SETUP.md` walks a non-expert from zero to live on both hosts, and every step was actually executed and verified
- ☐ First-run setup wizard in the panel guides configuration to completion
- ☐ `/docs/content-map.md` delivered: every visible string, image, and link in the theme mapped to the exact admin screen and field that edits it, with no unmapped rows
- ☐ Banners and sliders fully manageable (desktop + mobile image, copy, CTA, link, order, schedule, on/off) without code
- ☐ Header, menus, footer columns/links, social URLs, and the copyright text are all editable in the panel; the year updates itself
- ☐ Every UI microcopy string — button labels, empty states, validation messages, 404 copy — editable in the string editor
- ☐ CI lint rule fails the build on any hardcoded user-facing string in a component
- ☐ Every storefront page verified at 320–2560 px in both orientations with zero horizontal scroll, clipping, or overlap
- ☐ Screenshot set committed for the full device matrix (iPhone SE → 4K desktop, incl. foldable and iPad landscape)
- ☐ All touch targets $\geq 44 \times 44$ px; no functionality reachable only by hover; safe-area insets respected
- ☐ Admin panel fully usable on a phone: orders, status updates, invoice download, and product editing all work
- ☐ Wide tables collapse into cards on small screens instead of overflowing (storefront and admin)
- ☐ Pinch-zoom enabled, usable at 200% text zoom, `prefers-reduced-motion` honoured
- ☐ Emails render correctly on mobile Gmail/Apple Mail/Outlook; invoice PDF correct at A4 and Letter
- ☐ Redeploying or replacing the app code provably leaves all products, customers, orders, and invoices untouched (demonstrated on a seeded copy)

- ☐ Nightly automated backup of database and storage media, encrypted, off-site, with integrity verification and alerting on failure
- ☐ Backups screen in the panel: back up now, history, one-archive "download everything", guarded restore, scheduled exports
- ☐ CSV/JSON export and matching import for products, customers, and orders — a full rebuild from exports is possible
- ☐ Seed script and deploy pipeline cannot truncate, reset, or overwrite existing data; destructive migrations are labelled, flag-gated, and auto-backed-up
- ☐ A full restore drill was actually performed and reported (row counts, timing, screenshots of the restored store)
- ☐ `/docs/runbooks/upgrade.md` and `/docs/runbooks/restore.md` delivered, with staging-first upgrade steps and a code-rollback path that leaves data intact
- ☐ 30-day soft-delete/trash for products, categories, customers, pages, and media
- ☐ Mobile-number OTP verification fully built, provider-agnostic, and shipped disabled — the store is 100% functional with SMS off
- ☐ SMS adapters delivered for Twilio, Vonage, MessageBird, Plivo, Telnyx, AWS SNS, Infobip, WhatsApp Cloud API, a generic HTTP gateway (any local provider, configured in the panel), and a dev log adapter
- ☐ Email OTP / magic link is the free default verification channel and every SMS failure falls back to it without blocking a sale
- ☐ OTP codes hashed at rest, 6 digits, 5-minute expiry, 5-attempt limit, single-use, 60-second resend cooldown
- ☐ OTP endpoint rate-limited per number, per IP, per session, and per day, with CAPTCHA escalation and lockout — SMS-pumping fraud tested and blocked
- ☐ Monthly SMS spend cap, per-country enable list, live send counter and cost estimate in the panel; hitting the cap disables SMS instead of billing me
- ☐ OTP required only where I switch it on (COD orders, high-value orders, phone change, phone login) — each an independent toggle
- ☐ SMS templates editable per locale with segment/cost counter; `sms_log` visible in the panel with a working "send test SMS" button